

JOB APPLICATION TRACKER

1. Project Description

The Job Application Tracker is a backend-based application developed using Node.js, Express.js, and MongoDB that enables users to efficiently manage and track job and internship applications. The system stores application details such as company name, job role, salary, location, application date, and current application status. The application operates entirely through REST APIs and does not include a frontend interface. Users interact with the system using Postman and Swagger UI for API testing and documentation.

The project demonstrates practical implementation of backend development concepts including database integration, API development, data validation, aggregation operations, and API documentation.

2. Project Objective

The primary objective of this project is to develop a centralized platform for managing job applications while learning modern backend development practices. The project aims to:

- Store and manage job application records efficiently.**
- Perform CRUD operations using REST APIs.**
- Demonstrate MongoDB database integration using Mongoose.**

- **Generate analytical insights using MongoDB Aggregation Framework.**
 - **Document APIs using Swagger UI.**
 - **Provide hands-on experience with Node.js and Express.js.**
-

3. Project Scope

The scope of the project includes:

- **Managing job application records.**
- **Searching applications by company name.**
- **Filtering applications based on application status.**
- **Generating application analytics and reports.**
- **Database seeding for testing purposes.**
- **API testing through Postman.**
- **Interactive API documentation through Swagger UI.**

The project is designed as a backend service and can later be extended with authentication, user management, frontend dashboards, and cloud deployment.

4. Problem Statement

Students and job seekers often apply to multiple companies simultaneously and struggle to keep track of application statuses, interview schedules, and outcomes.

Managing this information manually through spreadsheets or notes can lead to inefficiencies and missed opportunities.

The Job Application Tracker addresses this problem by providing a centralized backend system where all application-related information can be stored, updated, searched, and analyzed efficiently.

5. Tech Stack

Backend

- **Node.js**
- **Express.js**

Database

- **MongoDB**
- **Mongoose ODM**

API Testing

- **Postman**

Documentation

- **Swagger UI**
- **Swagger JSDoc**

Development Tools

- **Nodemon**
- **Dotenv**

6. Database Design

Database Name

jobApplicationTracker

Collection Name

applications

Application Schema

Field	Data Type	Description
companyName	String	Name of company
jobRole	String	Position applied for
location	String	Job location
salary	Number	Expected salary/stipend
status	String	Current application status
applicationDate	Date	Date of application
createdAt	Date	Record creation timestamp
updatedAt	Date	Record update timestamp

Status Values

- **Applied**
- **Shortlisted**
- **Interview Scheduled**
- **Rejected**

- **Selected**
-

7. Key Features

Application Management

- **Create Application**
- **View All Applications**
- **View Application by ID**
- **Update Application**
- **Delete Application**

Search and Filtering

- **Search by Company Name**
- **Filter by Application Status**

Analytics

- **Total Applications Count**
- **Status Distribution Analysis**
- **Company-wise Statistics**
- **Average Salary Analysis**
- **Monthly Application Trends**

Additional Features

- **MongoDB Database Integration**
- **Seed Data Generation**
- **Swagger Documentation**

- **RESTful API Architecture**

8. System Workflow

- 1. User sends API requests through Postman or Swagger UI.**
- 2. Express.js receives and processes the request.**
- 3. Controllers validate request data.**
- 4. Mongoose communicates with MongoDB.**
- 5. Requested operation is executed.**
- 6. Database returns the result.**
- 7. Server sends JSON response back to the client.**
- 8. Analytics APIs use MongoDB Aggregation Pipeline to generate reports.**

Workflow Diagram

Client (Postman)

|

v

Express Server

|

v

Controllers

|

v

Mongoose ODM

|

v

MongoDB Database

|

v

JSON Response

9. API Modules

Application Module

Method	Endpoint	Purpose
POST	/api/applications	Create application
GET	/api/applications	Get all applications
GET	/api/applications/:id	Get application by ID
PUT	/api/applications/:id	Update application
DELETE	/api/applications/:id	Delete application

Search Module

Method	Endpoint	Purpose
GET	/api/applications/search	Search company

Method	Endpoint	Purpose
GET	/api/applications/status/:status	Filter by status

Analytics Module

Method	Endpoint	Purpose
GET	/api/analytics/total	Total applications
GET	/api/analytics/status-stats	Status statistics
GET	/api/analytics/company-stats	Company statistics
GET	/api/analytics/average-salary	Salary analysis
GET	/api/analytics/monthly-trends	Monthly trends

10. Learning Outcomes

Through this project, the following concepts were learned and implemented:

- Node.js backend development**
- Express.js routing and middleware**
- MongoDB database operations**
- Mongoose schema design**
- CRUD API development**
- RESTful architecture**
- MongoDB Aggregation Framework**
- API testing using Postman**

- **API documentation using Swagger UI**
 - **Database seeding techniques**
 - **Environment variable management**
 - **Project structuring and modular development**
-

11. Conclusion

The Job Application Tracker successfully demonstrates the implementation of a backend application using Node.js, Express.js, and MongoDB. The system provides efficient management of job application records while showcasing important backend development concepts such as CRUD operations, aggregation pipelines, API documentation, and database integration. The project serves as a practical example of modern REST API development and establishes a strong foundation for future enhancements such as authentication, frontend integration, and cloud deployment.

```

> job-application-tracker@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (2) from .env // tip: $ custom filepath { path: '/custom/path/.env' }
Server Running On Port 8000
MongoDB Connected: 127.0.0.1
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (2) from .env // tip: < auth for agents [www.vestaauth.com]
Server Running On Port 8000
MongoDB Connected: 127.0.0.1
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (2) from .env // tip: ♦ secrets for agents [www.dotenvx.com]
Server Running On Port 8000
MongoDB Connected: 127.0.0.1

```

Create Application

POST `http://localhost:8000/api/applications`

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

● none ● form-data ● x-www-form-urlencoded ● **raw** ● binary **JSON** ▾

```

1  {
2    "companyName": "Google",
3    "jobRole": "Software Engineer Intern",
4    "location": "Bangalore",
5    "salary": 45000,
6    "status": "Applied"
7  }

```

Body Cookies Headers (8) Test Results

Pretty Raw Preview Visualize **JSON** ▾ ⇌

```

1  {
2    "companyName": "Google",
3    "jobRole": "Software Engineer Intern",
4    "location": "Bangalore",
5    "salary": 45000,
6    "status": "Applied",
7    "_id": "6a36c7c6f8079e7da26cfb28",
8    "applicationDate": "2026-06-20T17:03:02.959Z",
9    "createdAt": "2026-06-20T17:03:02.964Z",
10   "updatedAt": "2026-06-20T17:03:02.964Z",
11   "__v": 0
12 }

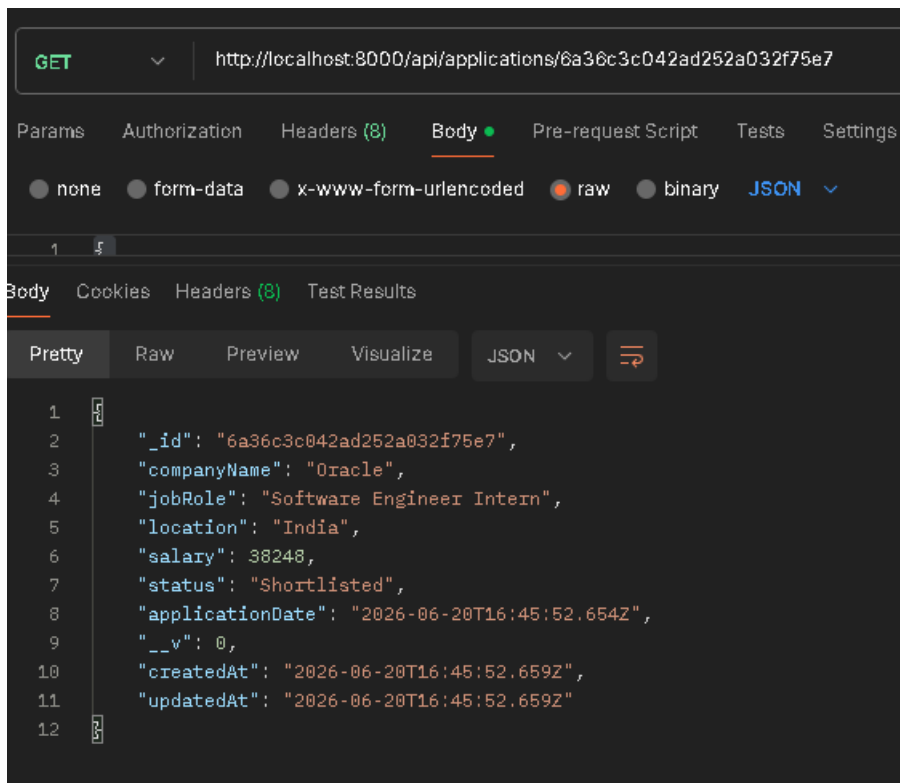
```

Get All application

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/applications`. The response is displayed in JSON format, showing a list of three application records. Each record contains fields for `_id`, `companyName`, `jobRole`, `location`, `salary`, `status`, `applicationDate`, `__v`, `createdAt`, and `updatedAt`.

```
1  {
2    "_id": "6a36c3c042ad252a032f75e7",
3    "companyName": "Oracle",
4    "jobRole": "Software Engineer Intern",
5    "location": "India",
6    "salary": 38248,
7    "status": "Shortlisted",
8    "applicationDate": "2026-06-20T16:45:52.654Z",
9    "__v": 0,
10   "createdAt": "2026-06-20T16:45:52.659Z",
11   "updatedAt": "2026-06-20T16:45:52.659Z"
12 },
13 {
14   "_id": "6a36c3c042ad252a032f75e8",
15   "companyName": "Microsoft",
16   "jobRole": "Software Engineer Intern",
17   "location": "India",
18   "salary": 54946,
19   "status": "Selected",
20   "applicationDate": "2026-06-20T16:45:52.655Z",
21   "__v": 0,
22   "createdAt": "2026-06-20T16:45:52.660Z",
23   "updatedAt": "2026-06-20T16:45:52.660Z"
24 },
25 {
26   "_id": "6a36c3c042ad252a032f75e9",
27   "companyName": "Microsoft",
28   "jobRole": "Software Engineer Intern",
29   "location": "India",
30   "salary": 51152,
31   "status": "Selected",
32   "applicationDate": "2026-06-20T16:45:52.655Z",
33   "__v": 0,
34   "createdAt": "2026-06-20T16:45:52.660Z",
35   "updatedAt": "2026-06-20T16:45:52.660Z"
36 },
37 }
38 }
```

application by id



GET ▼ http://localhost:8000/api/applications/6a36c3c042ad252a032f75e7

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary JSON ▼

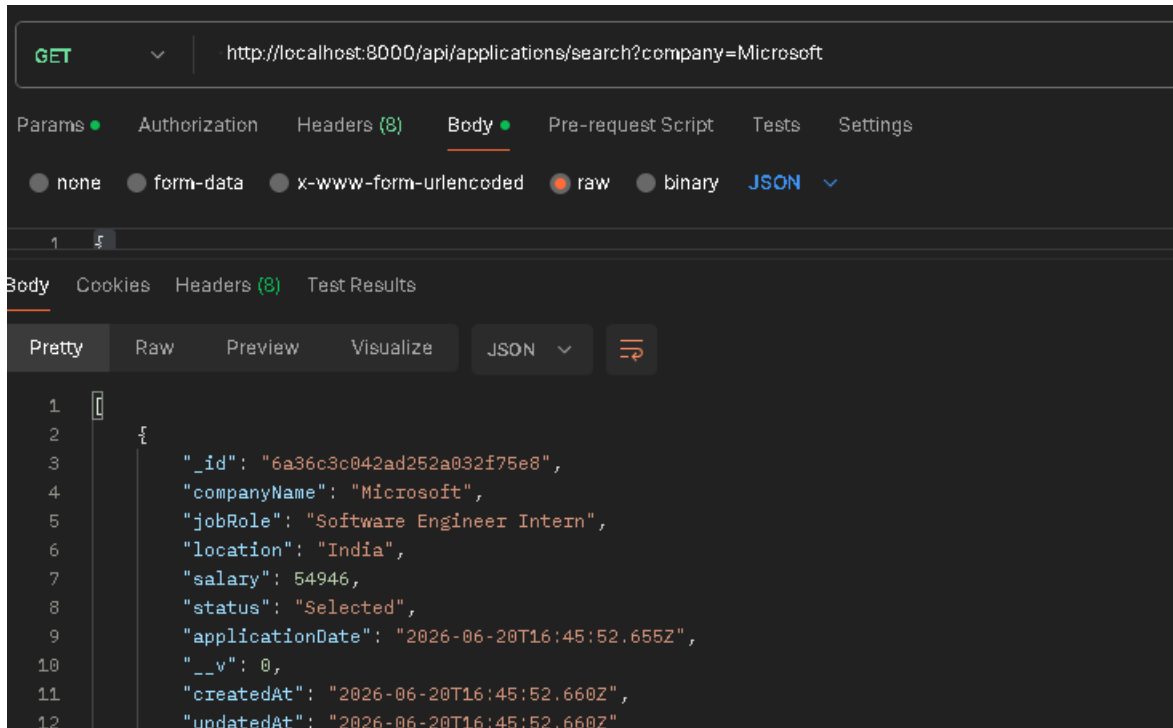
1 5

Body Cookies Headers (8) Test Results

Pretty Raw Preview Visualize JSON ▼ ≡

```
1 {
2   "_id": "6a36c3c042ad252a032f75e7",
3   "companyName": "Oracle",
4   "jobRole": "Software Engineer Intern",
5   "location": "India",
6   "salary": 38248,
7   "status": "Shortlisted",
8   "applicationDate": "2026-06-20T16:45:52.654Z",
9   "__v": 0,
10  "createdAt": "2026-06-20T16:45:52.659Z",
11  "updatedAt": "2026-06-20T16:45:52.659Z"
12 }
```

Search



GET ▼ http://localhost:8000/api/applications/search?company=Microsoft

Params ● Authorization Headers (8) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary JSON ▼

1 5

Body Cookies Headers (8) Test Results

Pretty Raw Preview Visualize JSON ▼ ≡

```
1 {
2   "_id": "6a36c3c042ad252a032f75e8",
3   "companyName": "Microsoft",
4   "jobRole": "Software Engineer Intern",
5   "location": "India",
6   "salary": 54946,
7   "status": "Selected",
8   "applicationDate": "2026-06-20T16:45:52.655Z",
9   "__v": 0,
10  "createdAt": "2026-06-20T16:45:52.660Z",
11  "updatedAt": "2026-06-20T16:45:52.660Z"
12 }
```

Swagger UI

Swagger
powered by SMARTBEAN

Job Application Tracker API 1.0.0 OAS 3.0

Backend API for managing and tracking job applications using Node.js, Express.js and MongoDB

Servers
<http://localhost:8000>

Applications

- GET** `/api/applications` Get all applications
- POST** `/api/applications` Create a new application
- GET** `/api/applications/{id}` Get application by ID
- PUT** `/api/applications/{id}` Update application
- DELETE** `/api/applications/{id}` Delete application
- GET** `/api/applications/search` Search by company name
- GET** `/api/applications/status/{status}` Filter applications by status

Analytics

- GET** `/api/analytics/total` Get total applications
- GET** `/api/analytics/status-stats` Status statistics
- GET** `/api/analytics/company-stats` Company statistics
- GET** `/api/analytics/average-salary` Average salary analytics
- GET** `/api/analytics/monthly-trends` Monthly application trends

MongoDB

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (5)

Search connections

FirstCollection

learning MDB

myproject

Project1

- admin
- ai_meeting_vault
- college
- config
- ecommerce
- firstdb
- jobApplicationTracker
 - applications
- local
- productFeedbackDB
- secondbrain
- test

Project2

local applications +

Project1 > jobApplicationTracker > applications

Documents 51 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

location : "India"
salary : 41861
status : "Interview Scheduled"
applicationDate : 2026-06-20T16:45:52.657+00:00
__v : 0
createdAt : 2026-06-20T16:45:52.661+00:00
updatedAt : 2026-06-20T16:45:52.661+00:00

_id: ObjectId('6a36c3c042ad252a032f7617')

companyName : "Adobe"
jobRole : "Software Engineer Intern"
location : "India"
salary : 48324
status : "Applied"
applicationDate : 2026-06-20T16:45:52.657+00:00
__v : 0
createdAt : 2026-06-20T16:45:52.661+00:00
updatedAt : 2026-06-20T16:45:52.661+00:00

_id: ObjectId('6a36c3c042ad252a032f7618')

companyName : "Oracle"
jobRole : "Software Engineer Intern"
location : "India"
salary : 63711
status : "Applied"
applicationDate : 2026-06-20T16:45:52.657+00:00
__v : 0
createdAt : 2026-06-20T16:45:52.661+00:00
updatedAt : 2026-06-20T16:45:52.661+00:00

_id: ObjectId('6a36c7c6f9079e7da26cfb29')

companyName : "Google"
jobRole : "Software Engineer Intern"
location : "Bangalore"
salary : 45000
status : "Applied"
applicationDate : 2026-06-20T17:03:02.959+00:00
createdAt : 2026-06-20T17:03:02.964+00:00
updatedAt : 2026-06-20T17:03:02.964+00:00
__v : 0